Retrieval-Augmented Generation: Concepts, Hallucination Mitigation and Malicious PyPI Package Detection

(draft: experiment results still to be filled in)

Trịnh Đình Hoàn 20261076M
Phạm Trung Dũng 20261080M
Đoàn Hải Yến 20252746M
Đỗ Tuấn Minh 20261057M
Hoàng Huy Chiến 20261069M

Abstract

Large language models (LLMs) often state wrong or unsupported content with confidence, a failure called hallucination. Retrieval-augmented generation (RAG) gives the model passages from an external knowledge base instead of relying on its parameters alone. This report covers two tasks: the basic concepts of RAG, and the use of RAG to mitigate hallucination. For the second task we run two linked studies on a shared code base. In the first, a ladder of six configurations (B0–B5) on the CRAG benchmark (1,335 public test questions, Llama-3-8B-Instruct) adds one component at a time: documents, snippets, more pages, hybrid BM25 + dense retrieval with reciprocal rank fusion, a cross-encoder reranker with a relevance threshold, a citation constraint, and a refusal gate; each step is scored with the CRAG labels accurate, hallucinated and missing and with truthfulness, by two independent graders. In the second, we reproduce and extend the study of Ibiyo et al., who found that RAG did not help detect malicious PyPI packages, by comparing no-RAG, RAG and RAG with few-shot examples over YARA rules, GitHub advisories and labelled `setup.py` files, and by adding hybrid retrieval, reranking, code-aware chunking and an evidence-grounding gate. The code for both studies is implemented; the full result tables are not yet filled in. **[TODO: Add the headline numbers once the runs are finished.]**

1 Introduction

LLMs are used for question answering, summarisation and code analysis, but they sometimes state wrong or unsupported content with the same confidence as correct content [2, 3]. This behaviour is called *hallucination*. It is more frequent for facts that are rare, recent or private, which a model has seen little or never [5], and for small models that store fewer facts [4].

Retrieval-augmented generation (RAG) [6, 7] adds an external knowledge base. For each question the system retrieves the most relevant passages and places them in the prompt, so the model can answer from text rather than from memory. Early work reported that retrieval reduces hallucination in knowledge-grounded dialogue [8].

RAG is not a guarantee. If retrieval returns irrelevant passages, models can be distracted by them [9]; if the passages do not contain the answer, models often answer anyway [10, 11]; and if the passages conflict with what the model believes, it may follow either side [12]. The study that motivates this project, by Ibiyo et al. [1], is a concrete case. They used RAG with YARA rules, GitHub Security Advisories and malicious `setup.py` files to classify PyPI packages, and found that RAG scored below zero-shot prompting and far below few-shot fine-tuning.

Our repository README summarises this as "middling results" for RAG and asks *why*: poor retrieval, hallucination by the LLM, or a mismatch between the retrieved context and the task.

Project tasks.

1. **Basic concepts in RAG** (Section 2).
2. **Using RAG to mitigate hallucinations** (Sections 4 to 6), through two studies that share one code base:
   - **E1, a CRAG ladder.** Which component of a RAG pipeline actually reduces hallucination on general question answering? Each of six configurations changes exactly one thing relative to the previous one.
   - **E2, malicious PyPI package detection.** Can the RAG design be improved for the reference task, and does an evidence-based gate against unsupported answers help?

Status. The code for both studies is in the project repository. This draft describes the method as implemented there; the result tables are placeholders because the final runs are not yet recorded in the repository. Text that needs a team decision or a number is marked **[TODO: like this]** or placed in a yellow box.

Contributions.

- A concise introduction to the RAG pipeline, its variants and its evaluation (Section 2).
- A controlled ladder of RAG configurations on CRAG with a fixed 4,000-token document budget, two independent graders and a calibrated refusal threshold (Section 5.2).
- A storage-agnostic RAG detector for malicious PyPI packages with no-RAG, RAG and few-shot modes, and a plan for hybrid retrieval, reranking, code-aware chunking and a grounding gate (Section 5.4).
- A diagnosis of where RAG helps and where it does not (Sections 6 and 7). **[TODO: Confirm after results.]**

2 Basic concepts in RAG

2.1 Hallucination in LLMs

A hallucination is generated content that is unfaithful to a source or to the facts [2]. Two kinds are usually separated. *Faithfulness* hallucinations contradict or go beyond the provided input (for example, a summary that adds a claim not in the document); *factuality* hallucinations contradict world knowledge [3, 13]. In a RAG system the two are connected: the provided input is the retrieved context, so an answer can be faithful to the context but wrong (the context was the wrong passage), or unfaithful to a correct context. We therefore need to track both.

Typical causes are gaps in the model's parametric knowledge (long-tail, recent or private facts [5]), limited model size [4], and decoding that always produces an answer even when none is known. These causes explain why giving the model external text is attractive.

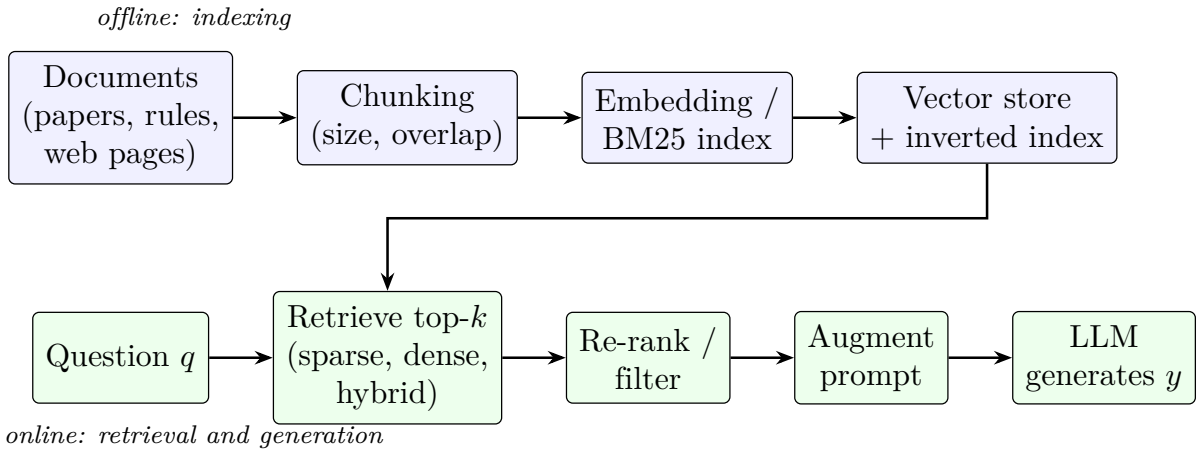


Figure 1: The standard RAG pipeline. Blue boxes run once when the knowledge base is built; green boxes run for every question.

2.2 The RAG pipeline

A RAG system answers a question q using a knowledge base K of text chunks. Figure 1 shows the standard pipeline, with an offline indexing phase and an online query phase.

Indexing. Documents are split into chunks, because a whole document is too long for the prompt and too coarse for matching. Chunk size and overlap are design choices: small chunks match precisely but lose context, large chunks keep context but add noise. Each chunk is stored in a sparse index, a dense vector index, or both.

Retrieval. Given q, the retriever returns the k highest-scoring chunks.

- *Sparse retrieval* such as BM25 [14] scores word overlap and is strong for names, identifiers and rare terms.
- *Dense retrieval* embeds q and each chunk with an encoder (for example DPR [15] or BGE-M3 [16]) and ranks by cosine similarity, so it matches paraphrases. Exact or approximate nearest-neighbour search, for instance FAISS [17], makes this fast.
- *Hybrid retrieval* combines both rankings, for example with reciprocal rank fusion, $\mathrm{RRF}(c) = \sum_r 1/(60 + \mathrm{rank}_r(c))$ [18], which needs no score calibration.
- *Re-ranking and filtering* apply a stronger model to the top candidates, or drop candidates judged irrelevant (Section 2.3).

Augmentation. The retrieved chunks are inserted into a prompt template together with an instruction such as "answer only from the documents; if they do not contain the answer, say you do not know". The order and number of chunks matter, since models use information in the middle of a long context less reliably [19].

Generation. The LLM produces the answer conditioned on the question and the chunks. In the original formulation [6], the retriever $p_\eta(z \mid q)$ and the generator $p_\theta(y \mid q, z)$ are combined as

$$p(y \mid q) \approx \sum_{z \in \text{top-}k\,(p_\eta(\cdot \mid q))} p_\eta(z \mid q)\, p_\theta(y \mid q, z). \tag{1}$$

In practice, with a frozen LLM, the chunks are simply concatenated in the prompt and the model is decoded once.

2.3 RAG variants

Gao et al. [7] distinguish three levels. *Naive RAG* is the pipeline above. *Advanced RAG* adds steps before retrieval (query rewriting), after retrieval (re-ranking, compression) or in indexing (better chunking, metadata). *Modular RAG* treats these steps as interchangeable modules and allows loops and routing. Two variants are directly relevant to hallucination:

- **Corrective RAG (CRAG)** [20] adds a lightweight *retrieval evaluator* that grades the retrieved documents and, depending on the grade, uses them, discards them, or triggers another knowledge source before generation. Ibiyo et al. [1] use this idea to append only highly relevant rules or advisories to the prompt. (This method is different from the CRAG benchmark of [21], which we use for metrics in Section 2.6.)
- **Self-RAG** [22] trains the generator to decide when to retrieve and to critique its own output with reflection tokens. It needs fine-tuning, whereas the checks we consider are training-free.

2.4 Components used in this project

Besides the standard pipeline, our repository uses four components that target retrieval quality and hallucination directly.

- **Cross-encoder reranking.** A bi-encoder embeds the query and the passage separately; a cross-encoder reads them together and outputs a relevance score, which is slower but more accurate. We use it on the top candidates (the BGE reranker, `bge-reranker-v2-m3`), and, because its score lies in $[0, 1]$ and is comparable across questions (unlike an RRF score), we also use it as a *relevance threshold*.
- **Code-aware chunking (cAST).** Splitting source code by a fixed number of words can cut a function in half. Chunking along the abstract syntax tree keeps syntactic units together [45]. This is a configuration option for the PyPI detector (`pipeline.chunking: naive | cast`).
- **Citation constraint.** The prompt requires the model to answer only from numbered references and to end with the number of the reference used; if no reference supports an answer, it must say it does not know. The citation is removed before scoring.
- **Refusal gate and grounding check.** A *gate* decides before generation: if even the best retrieved passage scores below a threshold, the system answers "I don't know" without calling the model. A *grounding check* runs after generation and tests whether the answer is supported by the retrieved evidence, for example with a hallucination detector such as LettuceDetect [46] or the detectors of Section 2.6.

2.5 Why RAG can reduce hallucination, and how it can fail

Table 1 summarises the mechanisms and the failure modes. The right-hand column is the reason for the empirical study: each failure mode can be measured.

2.6 Evaluating RAG and hallucination

Evaluation has three layers.

Table 1: How each RAG ingredient is meant to reduce hallucination and how it can fail.

Ingredient	Intended effect	Typical failure mode
Knowledge base	Supplies facts the model lacks (long-tail, recent, private)	The answer is not in the KB, or the KB is not suited to the task (no standard malware KB exists, [1])
Retrieval	Finds the passage that contains the answer	Retrieval miss; near-duplicate or distractor passages rank higher
Augmented prompt	Tells the model to answer from the text and to refuse otherwise	Model ignores the instruction; irrelevant context distracts it [9]
Generation	Copies or paraphrases the supporting span	Copies a span that is in the context but answers a different question; follows wrong context over correct memory [12]
Post-hoc check	Withholds unsupported answers	Lowers coverage; grounded but wrong answers still pass [23]

- **Retrieval:** recall@k (is a chunk with the answer among the top k?) and mean reciprocal rank (MRR).
- **Answer quality:** exact match or token-level F_1 against gold answers, or an LLM judge when wording varies. LLM judges are cheap but biased [24, 25].
- **Hallucination and faithfulness:** detectors such as HHEM [26] and MiniCheck [27] score whether a premise supports a claim, and sampling-based methods use the consistency of several answers [28, 29]. Frameworks such as RAGAS [30] and ARES [31] combine these with LLM judges.

The CRAG benchmark [21] gives each answer one of three labels, accurate (+1), missing (0, the system abstained) or hallucinated (−1), and reports *truthfulness*, the mean score. Because abstaining is better than a wrong answer, this score rewards systems that know when not to answer. Selective-prediction views of abstention [32, 33] add the coverage–risk trade-off: refusing more questions lowers the error rate on the rest but also lowers helpfulness.

3 Related work

RAG as a hallucination remedy. Lewis et al. [6] introduced RAG for knowledge-intensive tasks, and Shuster et al. [8] showed that retrieval reduces hallucination in knowledge-grounded dialogue. Mallen et al. [5] found that retrieval helps most for rare facts that the model never memorised, while for popular facts the parametric memory can be better. Surveys of RAG [7] and of hallucination [2, 3] organise the field.

When RAG does not help. RGB [10], FaithEval [34], UAEval4RAG [35] and the sufficient-context analysis of Joren et al. [11] show that models answer from insufficient context instead of abstaining. Shi et al. [9] and ClashEval [12] show that irrelevant or misleading context is often followed, and Liu et al. [19] show a position effect in long contexts.

Detection and abstention. Grounding detectors (HHEM [26], MiniCheck [27], NLI-based filters [36]) test whether a claim is supported by a context; they measure faithfulness, which is not the same as correctness [23]. SelfCheckGPT [28] and semantic entropy [29] use the consistency of sampled answers. Conformal abstention [37] gives guarantees on the error rate, and Wen et al. [38] survey abstention. Corrective RAG [20] and Self-RAG [22] build the check into the pipeline, and LettuceDetect [46] is a span-level hallucination detector for RAG answers.

Table 2: Malicious-class results reported in [1] (their Table 1). The RAG rows use LLaMA-3.1-8B with the named knowledge base.

System	Precision	Recall	F_1
GPT-4o-mini (zero-shot)	0.98	0.78	0.86
LLaMA-3-70B (zero-shot)	0.52	0.66	0.59
LLaMA-3.1-8B + RAG (YARA rules)	0.48	0.55	0.52
LLaMA-3.1-8B + RAG (GitHub advisories)	0.39	0.64	0.48
LLaMA-3.1-8B + RAG (malicious `setup.py`)	0.35	0.58	0.44

RAG in security and software engineering. Several studies add RAG to security tasks. LProtector [39] combines GPT-4o with CWE rules and code snippets for vulnerability detection, Vul-RAG [40] builds a knowledge base from CVEs, and Yu [41] applies RAG to smart-contract vulnerabilities; these report gains from retrieval. For code, cAST [45] chunks along the syntax tree to improve retrieval. The closest work to ours in spirit is Ibiyo et al. [1], summarised next.

3.1 Reference study: RAG for malicious PyPI package detection

Ibiyo et al. [1] classify PyPI packages as benign or malicious from four inputs: the package name, its file list, the `setup.py` content and an AST-based description of the code. They build three knowledge bases, embedded with `text-embedding-ada-002` in a PostgreSQL vector store: 6,220 YARA Forge rules [42] translated into plain text by an LLM, 3,474 GitHub security advisories (none specific to malware for PyPI), and `setup.py` files of malicious packages. A CRAG-style retrieval evaluator filters the retrieved documents before they reach the prompt. The dataset has 1,242 malicious packages from the DataDog collection [43] and 3,752 benign ones.

Table 2 lists the main comparison for the malicious class. All three RAG variants are worse than zero-shot GPT-4o-mini, and the authors conclude that no single knowledge base is sufficient, that fine-tuning (precision, recall and F_1 above 0.95 after fine-tuning) is the most effective approach, and that the RAG technique rather than the data source seems to be the problem. They call for better retrieval models, standardised malware knowledge bases and hybrid fine-tuning plus retrieval.

What we take from it. The study is a classification task, so its failures show up as false positives and false negatives, not as free-text hallucinations. Still, it motivates three points for this report: (i) RAG results must be compared with a no-retrieval baseline, not assumed to be better; (ii) the knowledge base and the retriever must be evaluated, not only the generator; and (iii) a negative result is informative if the failure is analysed. Its structure (research questions, comparison of knowledge bases, discussion and threats to validity) also guides ours.

4 Problem formulation

4.1 Tasks and research questions

4.2 Formal setting

A question or input q is given with a knowledge base $K = \{c_1, \ldots, c_M\}$ of chunks. A frozen generator G produces

$$C = \mathrm{Retr}_k(q, K), \qquad y = G(q, C), \tag{2}$$

Table 3: Project tasks, where they are covered, and research questions.

Task / study	Section	Research question
1. Basic concepts	Section 2	Complete in this draft.
2. Mitigation, E1 (CRAG ladder)	Section 5.2	**RQ1** Does adding documents reduce hallucination? **RQ2** Which component (snippets, more pages, hybrid retrieval, reranking and threshold, citation constraint, refusal gate) gives the improvement? **RQ3** What does it cost in missing answers?
2. Mitigation, E2 (PyPI)	Section 5.4	**RQ4** Does RAG help detect malicious packages compared with the zero-shot baseline, and with few-shot examples? **RQ5** Why does it fail when it fails: retrieval quality, LLM hallucination, or context mismatch? **RQ6** Do hybrid retrieval, reranking, cAST chunking and a grounding gate improve it?

where $C = \emptyset$ for the no-retrieval baseline. Optional components may *filter* C (a relevance threshold), *refuse* before generation (a gate on the best passage score) or *verify* y after generation (a grounding check). Reference answers are used only for scoring.

4.3 Labels and metrics

E1 (CRAG). Following the benchmark [21], each answer gets one of three labels: *accurate* (+1), *hallucinated* (−1) or *missing* (0; the model declined or said it does not know). With Acc, Hal and Miss the shares of the labels,

$$\text{truthfulness } T = \text{Acc} - \text{Hal}. \tag{3}$$

Penalising a hallucination more than a refusal is deliberate: saying "I don't know" is better than a wrong answer. In our repository the shares and T are reported in percent. Two independent graders produce the labels (Section 5.3), so every result has two versions.

E2 (PyPI). The detector outputs `malicious`, `benign` or `unknown`; `unknown` is counted as wrong. The repository implements accuracy and balanced accuracy (the mean recall of the classes, important because benign packages outnumber malicious ones); precision, recall, F_1 and false-negative rate are not implemented yet and are needed to compare with the reference paper. We also plan to measure the hallucination of the *explanation*: lines quoted as suspicious that do not appear in the code, and claims that the retrieved evidence does not support.

Decide: hallucination definition for E2

[TODO: Choose.] A classification error is not by itself a hallucination. Options: (a) report only classification metrics (as in [1]); (b) additionally check that every line in `suspicious_lines` occurs verbatim in the input code, which is cheap and objective; (c) use a grounding detector on the `reason` text against the retrieved evidence.

5 Method and system design

This section describes what is implemented in the repository. Table 4 states what runs today and what is still planned, so that the report does not claim more than the code does.

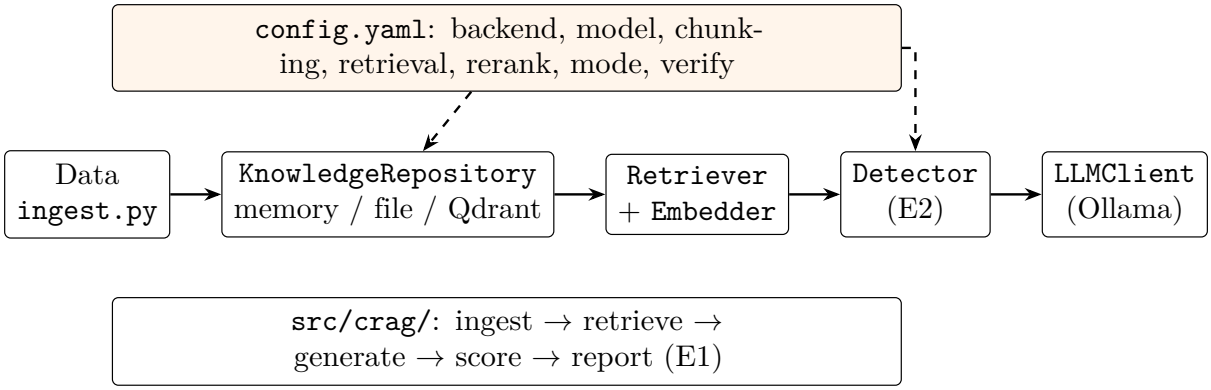


Figure 2: Module layout. Dashed arrows show configuration. The CRAG ladder (E1) has its own retrieval and generation scripts and shares only the evaluation code.

5.1 Architecture

The code base is organised so that data storage, retrieval, the LLM and the business logic can be changed independently (Figure 2).

- **Storage abstraction.** A `KnowledgeRepository` interface (`add`, `add_many`, `get`, `all`, `search`, `count`, `clear`) has three backends: in-memory, a JSON file, and Qdrant. The backend is chosen by `storage.backend` in `config.yaml`; all backends share one cosine implementation, and unit tests check that they return identical results.
- **Embedding and retrieval.** `Embedder` (Ollama `bge-m3`, or an offline hashing embedder for tests) and `Retriever` (vector search with an optional score threshold).
- **LLM client.** `LLMClient.generate` over Ollama; it strips reasoning blocks and warns when an answer is truncated at `max_tokens`, since truncated answers confound evaluation.
- **Detector.** Depends only on the `Retriever` and `LLMClient` interfaces, so it does not know where the knowledge base lives.
- **Feature flags.** `config.yaml` holds `chunking` (naive | cast), `retrieval` (vector | hybrid), `rerank`, `mode` (no_rag | rag | rag_fewshot) and `verify`, so each component can be switched on for an ablation without code changes.

5.2 E1: the CRAG ladder

Data and model. We use the public test split of CRAG [21]: 1,335 questions over five domains with web pages retrieved for each question. The generator is `llama3:8b-instruct-fp16` [48] run locally through Ollama, so its numbers can be compared with the Llama-3-8B row of the benchmark paper ([**TODO: confirm table number**]). The prompts are taken from the benchmark paper's appendix.

Ladder. Each configuration changes exactly one thing compared with the previous one, and the document budget stays at 4,000 tokens at every step, so the only thing that changes is *what* fills that budget (Table 5).

Retrieval (B2–B5). Retrieval runs once and its output is shared by B2–B5. Page text is split into chunks of 220 words with 40 words of overlap (at most 60 chunks per page). Two rankings are produced: BM25 ($k_1 = 1.5$, $b = 0.75$, implemented in the repository) and dense similarity

Table 4: Implementation status in the repository at the time of writing.

Component	Status	Note
Storage backends, config, `Document`	Implemented	Unit tests for the backends and the factory
Embeddings, vector retriever	Implemented	Hybrid and rerank in `src/retriever.py` are not wired in (`retrieval: hybrid` raises an error)
LLM client, detector (no_rag, rag, rag_fewshot)	Implemented	JSON verdict; unreadable output → `unknown`
Ingest of YARA, GHSA, PyPI	Implemented	Data are not stored in git; `setup.py` `scripts/fetch_data.py`
CRAG ladder B0–B5 (retrieve, generate, score, report)	Implemented	Needs GPUs and a large download
Retrieval experiments S1–S7	Demo only	Own `Document`/`Hit` classes; not connected to the repository
Precision, recall, F_1, FNR for E2	Not implemented	Only accuracy and balanced accuracy
Grounding gate (`verify`)	Config flag only	No implementation found in `src/`
cAST chunking in the detector	Planned	Option in the config

Table 5: The configurations of the CRAG ladder.

Step	Change from the previous step	Source
`B0`	No documents (the "LLM only" row)	Benchmark paper
`B1`	Concatenate the full text of 5 pages in their original order	Team variant, control
`B1.1`	Full text → search *snippets*	Task 1 of the benchmark
`B1.2`	5 pages → 50 pages	Task 3, without the mock knowledge graph
`B2`	Selection by chunking, BM25 + bge-m3, **RRF**	Team
`B3`	Add **cross-encoder** reranking and a relevance threshold of 0.05	Team
`B4`	Add a **citation constraint** `[n]`	Team
`B5`	Add a system-level **refusal gate**	Team

with `bge-m3`. They are fused with reciprocal rank fusion, $\mathrm{RRF}(c) = \sum_r 1/(60 + \mathrm{rank}_r(c))$, and the top 40 chunks go to the cross-encoder `bge-reranker-v2-m3`, which gives each chunk a score in $[0, 1]$. RRF is used because BM25 scores are unbounded and cosine scores lie in $[-1, 1]$, so raw scores cannot simply be added.

What fills the budget. B2 fills the 4,000 tokens with chunks ordered by RRF score. B3 orders by rerank score and drops chunks below the threshold, so it may use fewer tokens or none. B4 numbers the chunks and uses the citation prompt (Section A). B5 adds a gate: if the best rerank score is below the threshold, the model is not called and the answer is "I don't know".

Choosing the threshold from data. The threshold is not tuned on the final scores. `calibrate.py` checks, for every question, whether a normalised gold answer occurs in the retrieved chunks, and tabulates how this probability changes with the best rerank score. The threshold (0.05) is placed where that probability collapses, because below it the model can only invent an answer. The same value is used for B3, B4 and B5.

Generation settings. Temperature 0.1 and top-p 0.9 (the benchmark paper does not state a temperature; 0.1 follows the repository's sample code), at most 75 new tokens (the benchmark

scores the first 75 tokens, so a lower cap would inflate the hallucination rate unevenly across steps), context window 8,192 tokens, seed 1. References are clipped with the Llama-3 tokenizer so the 4,000-token budget is exact.

5.3 Scoring

Two independent scoring modes run on the same generation file, so the judge or the label definition can be changed without calling the generator again. The official scoring code did not run unchanged, so the team re-implemented it. Both modes trim the answer to its first 75 tokens, normalise it in the SQuAD style (lower case, no punctuation or articles) and use an LLM judge from a different model family than the generator, to avoid self-preference bias.

- **Mode `crag` (benchmark style, for comparison with the paper).** If the answer contains "i don't know" → missing; else if it equals a gold answer after normalisation → accurate; otherwise the judge returns True or False (→ accurate or hallucinated).
- **Mode `three` (control).** After an exact match, the judge itself chooses among accurate, hallucinated and missing, using the competition's broader definition of "missing" (including apologies, empty output and requests for clarification) and the SimpleQA rule that a hedged wrong answer is still hallucinated.

Answers the judge cannot parse are counted separately and scored like missing.

5.4 E2: malicious PyPI package detection

Task and knowledge base. The input is the source of a package file (for example `setup.py`). The knowledge base follows the reference study [1]: YARA rules, GitHub Security Advisories (GHSA), and `setup.py` files of benign and malicious packages split into train and test. The data are fetched by `scripts/fetch_data.py` and are not stored in git.

Detector. `MaliciousCodeDetector.detect(code)` builds a prompt and asks the LLM (default `qwen2.5:7b` [47], temperature 0.1, 512 tokens) for a JSON object with `label` (malicious or benign), a short `reason`, and `suspicious_lines` copied from the code. Code longer than 8,000 characters keeps its head and tail, and each evidence snippet is limited to 800 characters. The verdict also stores the `evidence` actually put in the prompt and the raw answer, so errors can be traced. Three modes are compared:

- `no_rag`: zero-shot baseline;
- `rag`: the top-k retrieved evidence ($k = 5$) is added to the prompt;
- `rag_fewshot`: as `rag`, plus two fixed labelled examples (one malicious, one benign).

The prompt tells the model to base its answer only on the code and not to claim behaviour that is not there.

Planned improvements (RQ6). Each is a flag in `config.yaml` and is added one at a time: hybrid BM25 + dense retrieval with RRF, cross-encoder reranking, cAST chunking, and a grounding gate (`verify`) that rejects an answer whose reason or suspicious lines are not supported by the code or the retrieved evidence.

Diagnosis (RQ5). To tell retrieval errors from generation errors we plan a *context-sufficiency* analysis (does the retrieved evidence contain what is needed?) and an *evidence-localisation* check (does the model point to the right lines?), as in Section 6.

Two cautions from the README

(1) `ingest_all` loads both the train *and* test package files into the knowledge base. When accuracy is measured on the test split, retrieval can return the very sample being judged together with its label. For E2 the knowledge base must be built from the **train split only**; the final method section should say that this was done. (2) The file backend rewrites its whole JSON file on every `add`; for the full dataset (about 190k documents) use the memory or Qdrant backend, or `add_many`.

5.5 Retrieval experiments S1–S7

A separate package, `experiments/p3_retrieval`, holds seven cumulative retrieval scenarios, each adding one variable to the previous one (Table 6). Each ships its own ablation runner with a stop condition (it exits with an error if the scenario does not beat the previous one). They run on a small demo corpus with a hashing embedder and a stub reranker, so their numbers show structure, not quality. They are not yet connected to `KnowledgeRepository`, and the PyPI knowledge base has not yet been plugged in. In this report they serve as a catalogue of candidate retrieval improvements; the ones we adopt are those implemented in E1 (hybrid, rerank) and the configuration flags of E2.

Table 6: Retrieval scenarios of the P3 package (from the repository).

ID	Adds	Relevance here
S1	dense + BM25 + RRF (hybrid)	Used in E1 (B2)
S2	structure-aware chunking, atomic blocks, enrichment header	Related to cAST (E2)
S3	cross-encoder reranking (replace or RRF fusion)	Used in E1 (B3)
S4	metadata pre-filter (scope, version)	Not needed
S5	query understanding (diacritics, lexicon)	Vietnamese input; not needed
S6	adaptive-k, MMR, U-shaped ordering	Candidate for E2
S7	calibration and a four-level answer decision (may-assert)	Related to the gate (B5)

6 Experimental evaluation

No result files are committed to the repository yet, so this section gives the setup as it is configured and empty tables for the results. Cells marked TBD must be filled from `results/crag/metrics.json` (E1) and from the detector runs (E2).

6.1 Setup

[TODO: Fix which judge model was actually used and state it once; the README and the run scripts name different models.]

Table 7: Experimental setup as configured in the repository.

	E1: CRAG ladder	E2: PyPI detection
Data	CRAG public test, 1,335 questions	Benign and malicious packages (train/test); YARA rules; GHSA
Generator	`llama3:8b-instruct-fp16`	`qwen2.5:7b` (default)
Embedding	`bge-m3`	`bge-m3` (Ollama)
Reranker	`bge-reranker-v2-m3`	planned
Context budget	4,000 tokens	top-5 evidence, 800 characters each
Judge	model of a different family from the generator (README: `gemma3:27b`; scripts: `gemma4:31b`)	not needed (labels are known)
Metrics	accuracy, hallucination, missing, truthfulness (two graders)	accuracy, balanced accuracy; precision, recall, F_1 (to add)
Hardware	one GPU $\geq$16 GB for generation; a second GPU for the judge	TBD

6.2 E1: CRAG ladder (placeholder)

Table 8: Placeholder. CRAG ladder, Llama-3-8B-Instruct, 1,335 questions, in percent. Mode `crag` is the benchmark-style grader and mode `three` the three-label control. $T = \text{Acc} - \text{Hal}$.

	Mode `crag`				Mode `three`			
Step	Acc	Hal	Miss	T	Acc	Hal	Miss	T
`B0` no docs	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B1` full text, 5 pages	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B1.1` snippets	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B1.2` 50 pages	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B2` RRF chunks	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B3` + rerank, threshold	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B4` + citation	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
`B5` + refusal gate	TBD	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Benchmark paper, Llama-3-8B	TBD	TBD	TBD	TBD	–	–	–	–

Analyses to report.

- **RQ1 (documents help?).** B0 vs. B1/B1.1: change in hallucination and truthfulness.
- **RQ2 (which component?).** Differences between consecutive steps (B1.1→B1.2: more pages; B1.1→B2: better selection; B2→B3: rerank and threshold; B3→B4: citation; B4→B5: gate). Because only one thing changes at a time, each difference is attributable to one component. **[TODO: Add paired bootstrap intervals over questions; the repository does not compute them yet.]**
- **RQ3 (cost).** Missing share per step. A step that lowers hallucination only by raising missing by the same amount gains nothing in T if the refused answers would have been correct; report how many refusals removed a hallucination.
- **By question type and domain.** CRAG labels each question with a domain, a type and static/dynamic; the generation files keep these fields, so the effect can be broken down (for example false-premise or dynamic questions).

Table 9: Placeholder. Calibration of the threshold (output of `calibrate.py`): for questions whose best rerank score falls in each range, the share whose gold answer occurs in the retrieved chunks.

Best rerank score	Questions	Gold answer in chunks (%)
$[-1.00, 0.01)$	TBD	TBD
$[0.01, 0.05)$	TBD	TBD
$[0.05, 0.10)$	TBD	TBD
$[0.10, 0.20)$	TBD	TBD
$[0.20, 0.40)$	TBD	TBD
$[0.40, 1.01)$	TBD	TBD

- **Grader agreement.** Compare modes `crag` and `three`; report where they differ (hedged answers, apologies).

6.3 E2: PyPI detection (placeholder)

Table 10: Placeholder. Malicious-class results on the test split. The reference study's numbers are in Table 2.

System	Acc	Bal. acc	Precision	Recall	F_1
`no_rag` (zero-shot)	TBD	TBD	TBD	TBD	TBD
`rag` (vector, naive chunks)	TBD	TBD	TBD	TBD	TBD
`rag_fewshot`	TBD	TBD	TBD	TBD	TBD
+ hybrid retrieval	TBD	TBD	TBD	TBD	TBD
+ reranking	TBD	TBD	TBD	TBD	TBD
+ cAST chunking	TBD	TBD	TBD	TBD	TBD
+ grounding gate	TBD	TBD	TBD	TBD	TBD

Analyses to report. (i) RAG vs. zero-shot and few-shot, compared with the reference study (RAG below zero-shot). (ii) Error source (RQ5): split errors into those where the retrieved evidence was insufficient and those where it was sufficient but the model still erred. (iii) Knowledge-base ablation: YARA only, GHSA only, labelled `setup.py` only, all (train split only, see the caution in Section 5.4). (iv) Hallucinated lines: share of `suspicious_lines` that do not occur in the input code. (v) Rate of `unknown` verdicts.

If the final runs are small

Run E2 on a balanced sample first (for example a few hundred packages per class), report balanced accuracy and bootstrap intervals, and state the sample size. A full run over the dataset needs the memory or Qdrant backend and is slow on a single GPU.

7 Discussion and threats to validity

7.1 Discussion (to be written after results)

Points to address

- E1: which step reduced hallucination most, and did it do so by making the model answer correctly or by making it refuse? Did more pages (B1.2) help or add noise? Did the threshold calibration pick a sensible point?

- E2: did RAG help against the zero-shot baseline? If not, is the cause retrieval (the evidence does not contain what is needed), the model (ignores or misreads it), or the task (no knowledge base of the right kind), as discussed in [1]?
- Common thread: in both studies, does a gate or a grounding check lower the error rate, and at what cost in coverage?
- Practical advice on when to build a RAG pipeline and when a stronger model, few-shot prompts or fine-tuning is the better choice.

7.2 Threats to validity

- **Internal.** Labels come from LLM judges, which have known biases [24, 25]; the two graders mitigate but do not remove this, and neither judge has been validated against human labels. The official CRAG scoring code was re-implemented, so absolute numbers may differ from the benchmark paper. The benchmark paper does not state a temperature, so our 0.1 is an assumption. In E2, if the test samples are in the knowledge base, retrieval leaks the label (Section 5.4).
- **External.** One generator per study (Llama-3-8B, Qwen2.5-7B), run locally in fp16 or the default Ollama quantisation. CRAG covers five domains of English web questions; results on code may differ.
- **Construct.** Truthfulness depends on the missing label counting as zero; a system that refuses often scores higher than one that answers often and is wrong half the time, which may not suit every application.
- **Conclusion.** Only one seed (1) and one threshold per ladder are used; with 1,335 questions, differences of one or two points may fall within the sampling error. **[TODO: Add intervals.]**

8 Conclusion and future work

This report introduced the basic concepts of RAG and described a method for studying RAG as a hallucination remedy: a controlled ladder of six configurations on the CRAG benchmark, and a reproduction and extension of a malicious PyPI package detector whose reference study found that RAG did not help. The method is implemented in the project repository; the results are still to be recorded.

Conclusion template

[TODO: Write after results.] (1) One sentence per research question with the key number. (2) The main lesson, for example which component of the ladder mattered and whether RAG helped on PyPI. (3) Limitations (Section 7). (4) Future work: wire hybrid retrieval and reranking into `src/retriever.py`, implement the grounding gate and the missing metrics, connect the S1–S7 package to `KnowledgeRepository`, test a second generator, and combine retrieval with fine-tuning as proposed in [1].

Acknowledgments

We thank the course instructor for the project topic and feedback.

References

[1] Motunrayo Ibiyo, Thinakone Louangdy, Phuong T. Nguyen, Claudio Di Sipio, and Davide Di Ruscio. Detecting malicious source code in PyPI packages with LLMs: Does RAG come in handy? In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)*, 2025. arXiv:2504.13769.

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[3] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[4] Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.

[5] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of ACL (Volume 1: Long Papers)*, pages 9802–9822, 2023.

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, pages 9459–9474, 2020.

[7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2023. arXiv:2312.10997.

[8] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. Retrieval augmentation reduces hallucination in conversation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 3784–3803, 2021.

[9] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. In *Proceedings of ICML*, volume 202 of PMLR, pages 31210–31227, 2023.

[10] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17754–17762, 2024.

[11] Hailey Joren, Jianyi Zhang, Chun-Sung Ferng, Da-Cheng Juan, Ankur Taly, and Cyrus Rashtchian. Sufficient context: A new lens on retrieval augmented generation systems. In *International Conference on Learning Representations*, 2025.

[12] Kevin Wu, Eric Wu, and James Zou. ClashEval: Quantifying the tug-of-war between an LLM's internal prior and external evidence. In *Advances in Neural Information Processing Systems 37 (Datasets and Benchmarks Track)*, 2024.

[13] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of ACL*, pages 1906–1919, 2020.

[14] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[15] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of EMNLP*, pages 6769–6781, 2020.

[16] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of ACL 2024*, pages 2318–2335, 2024.

[17] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.

[18] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of SIGIR*, pages 758–759, 2009.

[19] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[20] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024. arXiv:2401.15884.

[21] Xiao Yang et al. CRAG – comprehensive RAG benchmark. In *Advances in Neural Information Processing Systems 37 (Datasets and Benchmarks Track)*, pages 10470–10490, 2024.

[22] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024.

[23] Vaibhav Adlakha, Parishad BehnamGhader, Xing Han Lu, Nicholas Meade, and Siva Reddy. Evaluating correctness and faithfulness of instruction-following models for question answering. *Transactions of the Association for Computational Linguistics*, 12:681–699, 2024.

[24] Lianmin Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (Datasets and Benchmarks Track)*, pages 46595–46623, 2023.

[25] Anna Bavaresco et al. LLMs instead of human judges? A large scale empirical study across 20 NLP evaluation tasks. In *Proceedings of ACL (Volume 2: Short Papers)*, pages 238–255, 2025.

[26] Miaoran Li, Rogger Luo, and Ofer Mendelevitch. HHEM-2.1-Open. https://huggingface.co/vectara/hallucination_evaluation_model, 2024.

[27] Liyan Tang, Philippe Laban, and Greg Durrett. MiniCheck: Efficient fact-checking of LLMs on grounding documents. In *Proceedings of EMNLP*, pages 8818–8847, 2024.

[28] Potsawee Manakul, Adian Liusie, and Mark Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of EMNLP*, pages 9004–9017, 2023.

[29] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.

[30] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of EACL: System Demonstrations*, pages 150–158, 2024.

[31] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of NAACL (Volume 1: Long Papers)*, pages 338–354, 2024.

[32] Ran El-Yaniv and Yair Wiener. On the foundations of noise-free selective classification. *Journal of Machine Learning Research*, 11:1605–1641, 2010.

[33] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*, 2017.

[34] Yifei Ming, Senthil Purushwalkam, Shrey Pandit, Zixuan Ke, Xuan-Phi Nguyen, Caiming Xiong, and Shafiq Joty. FaithEval: Can your language model stay faithful to context, even if "the moon is made of marshmallows". In *International Conference on Learning Representations*, 2025.

[35] Xiangyu Peng, Prafulla Kumar Choubey, Caiming Xiong, and Chien-Sheng Wu. Unanswerability evaluation for retrieval augmented generation. In *Proceedings of ACL (Volume 1: Long Papers)*, pages 8452–8472, 2025.

[36] Ori Yoran, Tomer Wolfson, Ori Ram, and Jonathan Berant. Making retrieval-augmented language models robust to irrelevant context. In *International Conference on Learning Representations*, 2024.

[37] Yasin Abbasi Yadkori et al. Mitigating LLM hallucinations via conformal abstention, 2024.

[38] Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. Know your limits: A survey of abstention in large language models. *Transactions of the Association for Computational Linguistics*, 13:529–556, 2025.

[39] Ze Sheng, Fenghua Wu, Xiangwu Zuo, Chao Li, Yuxin Qiao, and Lei Hang. LProtector: An LLM-driven vulnerability detection system, 2024. arXiv:2411.06493.

[40] Xueying Du, Geng Zheng, Kaixin Wang, Jiayi Feng, Wentai Deng, Mingwei Liu, Bihuan Chen, Xin Peng, Tao Ma, and Yiling Lou. Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG, 2024. arXiv:2406.11147.

[41] Jeffy Yu. Retrieval augmented generation integrated large language models in smart contract vulnerability detection, 2024. arXiv:2407.14838.

[42] YARA HQ. YARA Forge – a rule-sharing platform for YARA. https://github.com/YARAHQ/yara-forge, 2024.

[43] Datadog Security Labs. Open-source dataset of malicious software packages. https://github.com/datadog/malicious-software-packages-dataset, 2023.

[44] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of ACL (Volume 2: Short Papers)*, pages 784–789, 2018.

[45] Yilin Zhang et al. cAST: Enhancing code retrieval-augmented generation with structural chunking via abstract syntax tree, 2025. **[TODO: verify authors and arXiv id.]**

[46] Ádám Kovács and Gábor Recski. LettuceDetect: A hallucination detection framework for RAG applications, 2025. **[TODO: verify arXiv id.]**

[47] Qwen Team. Qwen2.5 technical report, 2024.

[48] Abhimanyu Dubey et al. The Llama 3 herd of models, 2024.

A Prompts

Line breaks inside long lines were added for printing; the repository has the exact text.

B0, no documents (from the benchmark paper's appendix).

```
You are given a Question and the time when it was asked in the Pacific Time
Zone (PT), referred to as "Query Time". The query time is formatted as
"mm/dd/yyyy, hh:mm:ss PT". Your task is to answer the question in as few
words as possible.
Please follow these guidelines when formulating your answer:
1. If the question contains a false premise or assumption, answer "invalid question".
2. If you are uncertain or don't know the answer, respond with "I don't know".

### Question
```

```
{query}
### Query Time
{query_time}
### Answer
```

B1–B3, with references. As B0, with "Question, References and the time...", the sentence "The references may or may not help answer the question." and a block `### References {references}` before the answer.

B4–B5, citation constraint. As B1 with numbered references and two added rules:

```
3. Answer only from the references. End your answer with the number of the
   reference you used, in square brackets, for example: Paris [2].
4. If no reference supports an answer, respond with "I don't know".
```

Judge prompts. Mode `crag`: the judge receives the question, the ground truths and the prediction and replies with a JSON field "Accuracy" (True or False); a refusal is always False, and for an "invalid question" truth the prediction must be exactly that. Mode `three`: the judge replies with a JSON field "label" in {accurate, hallucinated, missing}, with definitions: missing = declines to answer (including apologies, empty output, requests for clarification); accurate = matches a ground truth, hedging allowed if the correct answer is fully stated; hallucinated = contradicts the truth or answers something else, including a hedged wrong answer. Numbers must match to the last significant figure.

E2 detector (system and task prompt).

```
System: You are a security analyst who reviews Python package source code
(setup.py, install hooks, modules) for malicious behaviour. Answer with a
JSON object only.

Task: Decide whether the code below is "malicious" or "benign".
Malicious signals: encoded or obfuscated payloads that get executed
(base64/zlib + exec/eval), downloading and running remote code, reading
credentials or environment secrets and sending them out, reverse shells,
persistence (cron, startup entries), and anything that runs automatically at
install time (custom install commands in setup.py). Ordinary packaging code
(metadata, dependencies, console_scripts) is benign.
Base your answer only on what is in the code; do not claim behaviour that is
not there.
Respond with JSON only:
{"label": "malicious" or "benign", "reason": "<one or two sentences>",
 "suspicious_lines": ["<exact line copied from the code>"]}
Use an empty list for suspicious_lines when the code is benign.
```

B Reproduction

Commands from the repository README (Python 3.11 or newer; Ollama installed).

Tests and detector.

```
pip install -r requirements.txt
make test
ollama pull qwen2.5:7b && ollama pull bge-m3
python scripts/fetch_data.py
python scripts/run_detector.py setup.py --mode rag --ingest
python scripts/run_detector.py setup.py --mode no_rag
```

CRAG ladder.

```
python scripts/download_data.py crag          # Task 1 and 2 (739 MB)
python scripts/download_data.py crag_task3    # only for B1.2 (7.7 GB)
python -m src.crag.ingest
python -m src.crag.retrieve --top-k 40 --rerank-k 40
python -m src.crag.calibrate
python -m src.crag.generate --config B4 --model llama3:8b-instruct-fp16 \
    --ref-source snippet --num-predict 75 --keep-threshold 0.05 \
    --gate-threshold 0.05
python -m src.crag.score --gen results/crag/gen_B4_*.jsonl --mode crag  --judge <judge>
python -m src.crag.score --gen results/crag/gen_B4_*.jsonl --mode three --judge <judge>
python -m src.crag.report                     # -> results/crag/metrics.json
```

`scripts/run_all_p75.sh` runs B0, B1.1, B1.2, B2, B3 and B1; `scripts/run_b45.sh` runs B4 and B5. Both resume after an interruption. Generation and judging can run on two GPUs in parallel, with two Ollama instances on different ports.

C Earlier pilot on SQuAD 2.0 (not part of the repository)

Before the project scope was fixed, an earlier course report ran a pilot on a balanced subset of SQuAD 2.0 (356 test questions, half unanswerable) with Phi-3-mini, hybrid BM25 + BGE-M3 retrieval and a post-hoc guardrail. It uses a different benchmark and generator from E1 and E2, so it is kept only as background and can be deleted.

Table 11: Pilot results (Phi-3-mini, SQuAD 2.0 subset). T is truthfulness as in Equation (3). C6 uses HHEM and answer relevance; $C6^{M}$ uses MiniCheck and relevance.

System	Acc	Hal	Miss	T	Cov.
No RAG	0.180	0.666	0.154	−0.486	0.846
RAG	0.419	0.365	0.216	0.053	0.784
C6	0.390	0.250	0.360	0.140	0.640
$C6^{M}$	0.399	0.174	0.427	0.225	0.573

Its main lesson was that most remaining hallucinations were spans copied from the correct paragraph in reply to a question that it does not answer, so a check that asks only whether the answer is *supported* by the context does not catch them. This is the reason to include a gate based on retrieval score and a citation constraint in E1 and to report hallucinated explanations in E2.

D Completion checklist

Delete this appendix before submission.

- ☐ Run B0–B5 in both scoring modes; fill Tables 8 and 9; add the benchmark paper's Llama-3-8B row.
- ☐ Fix the judge model name (README and scripts disagree) and state it once.
- ☐ Run E2 with a knowledge base built from the *train* split only; add precision, recall, F_1 and FNR to `src/evaluation.py`; fill Table 10.
- ☐ Decide which of hybrid, rerank, cAST and the gate will be implemented for E2 before the deadline, and remove the rest from the text.

☐ Verify the two references marked `TODO`; check the authors on the title page.

☐ Search for `TODO` and `TBD` (red) in all files.